

Semester Reflection Report:

Transitioning to Modern Enterprise Architectures

Focus Area: Frontend Development & Real-Time Queue Management.

I'm a student who moved from Java EE 7 to a modern React + Spring Boot stack this semester.

My capstone project focused on the Live Queue Tracker for our Witbank Central car wash.

I helped bridge the frontend (React) with the new REST backend (Spring Boot), so the UI could show live data from the server.

LiveQueue Tracker System:

Has a task board with three columns: Waiting In Queue, Washing, Ready for collection

The frontend talks to the backend to get bookings.

The UI is repeatedly asking the backend for new data every 5 seconds to see if anything has changed

The app handles errors gracefully. If the server goes down, the app logs "Connection lost" and keeps the UI responsive for users.

The system is made of (tech choices)

Frontend: React using functional components and hooks

State management: useState for the bookings array

Data fetching: axios with an async fetchQueue function

Real-time feel: useEffect with setInterval to poll every 5 seconds

function in useEffect to clear the interval when the component unmounts

Rendering: I used .filter() and .map() to split bookings by status and render them into the appropriate columns

UX polish: CSS animations like washing-anim and ready-pulse to visually show status changes

Data contract: Booking objects include fields such as id, plateNumber, carModel, and status

Key concepts

Decoupled frontend and backend speeds up development:

You can design the UI around a JavaScript Object Notation contract and use mock data while the backend is still being built.

State and rendering in React:

State is kept in the browser memory. Don't mutate arrays directly; use the setter function like setBookings to trigger re-renders.

Lifecycle and cleanup:

Background tasks, like intervals, must be cleaned up when the component unmounts to avoid memory leaks.

Functional programming in UI:

Using .filter() and .map() makes code readable and concise for splitting data into columns and turning it into UI elements.

API-first thinking:

Agree on a simple JavaScript Object Notation contract (for example, id, plateNumber, carModel, status) to guide frontend development and testing.

Moving from page reloads to state changes:

The UI updates in response to data changes, not by reloading the page.

Challenges and fixes

Challenge: Multiple intervals caused memory leaks.

Fix: Implemented a proper cleanup in useEffect: clearInterval(interval) when the component unmounts or when the effect re-runs.

Challenge: Server down or network issues.

Fix: Added error handling in fetchQueue to catch failures and log "Connection lost". The app remains usable even when the backend is temporarily unavailable.

Challenge: UI felt laggy during fetches.

Fix: Used `asynchronously` or `await` properly and kept the UI responsive while the data was loading.

Challenge: Keeping the UI in sync with the data model.

Fix: Kept the frontend using a clear JSON contract and updated the state through `setBookings` to trigger correct renders.

How this shift changed my work

From tightly coupled Java EE pages to API-driven development:

I could design the frontend against a predictable JSON contract and mock data, while the backend team worked on JPA repositories and REST controllers in parallel.

Frontend and backend could progress simultaneously, then be integrated later with confidence.

A clearer mental model:

I started thinking in terms of state changes, data contracts, and component lifecycles instead of page reloads and server-rendered HTML.

Improved testing mindset:

The decoupled approach made it easier to test the frontend in isolation using mock data, which sped up iteration cycles.

Code highlights conceptual

Data fetching loop high level:

`fetchQueue` retrieves an array of bookings from the backend.

`setBookings` updates the local state with the latest data.

State-driven rendering:

```
bookings.filter(b => b.status === "Washing").map(car => renderCar(car))
```

This keeps the UI declarative and easy to read.

Cleanup pattern:

```
useEffect(() => { const interval = setInterval(fetchQueue, 5000); return () =>
clearInterval(interval); }, []);
```

Data model (simple): a booking has id, plateNumber, carModel, status.

How the system is organized

Columns clearly labeled: Waiting, Washing, Ready for collection

Each vehicle is shown as a card or small ticket with key details (plate number, model, status)

Status-driven animations:

When a booking is in Washing, a washing-anim class shows a subtle motion.

When ready for pickup, a ready-pulse adds a gentle glow to the card.

Accessibility basics:

Focusable elements, clear headings, and readable contrast to help users with different needs.

Next steps and ideas for improvement

Move to real-time updates:

Consider WebSockets or Server-Sent Events for true live updates without polling.

Enhanced data handling:

Explore libraries like React Query or SWR for smarter data fetching, caching, and background updates.

UI enhancements:

Add loading spinners, retry options when the network fails, and a “no data” state when there are no bookings.

Error visibility:

Show user-friendly messages in the UI when the backend is unavailable, instead of only logging to the console.

Accessibility and testing:

Add Accessible Rich Internet Application attributes, keyboard navigation, and basic tests (unit tests for the data layer and integration tests for the UI flow).

Data growth considerations:

If the list grows large, think about pagination or virtual scrolling to keep the UI snappy.

Backend integration:

As the Spring Boot API evolves, update the frontend to reflect any changes in the booking contract (new fields, status values, or endpoints).

Conclusion

This journey is from a tightly bound Java EE environment to a decoupled React and Spring Boot stack has been a powerful learning experience. Building the LiveQueue Tracker taught me to embrace state, components, and data contracts. It gave me practical confidence to contribute to modern full-stack teams, delivering fast, resilient web applications. I now understand how to design UI that can work with a backend that's still evolving, while keeping users happy with a responsive, visually clear interface.

More importantly, I learned how to think like a modern frontend developer. I stopped thinking in terms of "page reloads" and started thinking in terms of "state changes" and "component lifecycles." While Java EE provided a robust foundation in backend logic and database transactions, this project gave me the practical, industry-relevant skills needed to build fast, responsive, and resilient web applications. I end this semester with a profound sense of accomplishment and a high degree of confidence in my ability to contribute to modern full-stack development teams.